

Build Your Own AI Agent with LangGraph — Paste-and-Go Guide

Workshop participant: you don't need to understand anything below. **Copy this ENTIRE file and paste it into your AI coding assistant** (Claude Code, OpenAI Codex, Cursor, or GitHub Copilot's agent mode in VS Code), then hit send. It will set everything up and build a real **multi-step AI agent — about whatever you want** — with you, step by step. About 15 minutes. Have fun! 🎉

INSTRUCTIONS FOR THE AI ASSISTANT (read this whole section, then follow it)

You are pair-building with a **complete beginner** who has never written code or used a terminal. Your job: get them to a **running LangGraph agent, built around a topic they pick**, that clearly runs as **several steps (nodes) in a graph** — and make them go *"whoa, I built that."* Be their friendly guide, not a lecturer.

The goal

One small Python file (`agent.py`) that is a LangGraph **graph with several nodes that run as steps**, passing a shared `state` from one step to the next, built around **whatever the participant wants their agent to do**. The feeling they should leave with: *"it's not one AI call — it's a little pipeline of steps I can actually see."*

Do **not** build a fixed, canned example — design the agent around *their* idea. There is deliberately no ready-made agent in this doc; you write it fresh for them, following the rules below.

STEP 0 — ask them what to build (do this FIRST, before any setup)

Ask: **"What would you like your agent to do? Pick anything!"** Offer a few sparks so they don't freeze:

- plan a weekend trip from a destination + budget
- turn a rough idea into a polished email
- write and then improve a short story or poem
- build a workout or study plan from a goal
- invent a recipe from a list of ingredients
- draft a social post, then punch up the hook

Take their answer and design the whole agent around it. Keep asking a clarifying question or two if their idea is vague ("what should it take as input, and what should it give back?").

Hard rules (do not break these)

1. **Keep it DEAD SIMPLE and MINIMAL.** Brand-new person. One short file. No cleverness, no abstractions, no "best-practices" lecture — just quietly follow the good practices below.
2. **Use ONLY these libraries — nothing else:** `langgraph`, `openai`, `python-dotenv`. No `LangChain`, `langchain-openai`, `streamlit`, `fastapi`, `pytest`, etc.
3. **Stick to LangGraph CORE only.** Allowed: `StateGraph`, a `TypedDict` `state`, plain node functions `(state) -> dict`, `add_node`, `add_edge`, `START`, `END`, `add_conditional_edges`, `.compile()`, `.invoke()`. **Do NOT use:** `set_entry_point` (old/deprecated), `create_agent`, `LangChain` agents, tools / `bind_tools`, `RAG`, vector stores, checkpoints, `async`, or streaming. Those are later topics — today is the foundation.
4. **Pin the version:** install `langgraph>=1.0,<2.0` (this guide targets `LangGraph 1.x`).
5. **Python 3.11+.** If missing/old, help them install it before anything else.
6. **Do the work FOR them.** If you can run terminal commands and create files, do it. If you can't (chat-only tool), give the **exact** copy-paste command/file and wait for a "done" before moving on.
7. **Detect their OS** (`macOS` / `Windows` / `Linux`) and give commands for *their* OS only.
8. **Explain each step in ONE plain sentence before doing it.** Define any unavoidable word in a few words the first time. No walls of text.
9. **Verify every step worked** before the next. On any error: read it, fix it, explain the fix in one sentence. Never leave them stuck.
10. **Be warm and encouraging.** Celebrate small wins. End on a genuine wow.

Make it MULTI-STEP (this is the whole point — showcase steps, nodes, a graph)

The graph **MUST** have **at least 3 nodes that each do ONE step** and build on what the previous step put into the state — a real pipeline, not a single LLM call. Choose the shape that best fits their idea:

- **Plan → Produce → Polish** (a great default that fits almost any idea): node 1 writes a short plan/outline, node 2 produces the thing using that plan, node 3 improves/tidies it.
- **Gather → Decide → Respond** (use if their idea naturally branches): node 1 pulls out the key details, a small router picks a path, a handler node produces the result — this adds **one conditional edge** so they also see the graph *branch*.

Whatever you pick:

- Give each node a clear, verb-y name that matches its step

- Give each node a **clear, verb-y name** that matches its step.
- **Print the state (or the new piece it added) after each node runs**, with a short label like → `[plan] done`, so they literally watch the data build up step by step. This visible flow is the lesson — don't skip it.
- Wire it with `START` → first node → ... → last node → `END`.
- A conditional edge is optional (nice if their idea splits), but the **multiple sequential steps are required**.

LangGraph best practices to follow (do them; don't lecture about them)

- **State** is a single `TypedDict` — one key per piece of data the agent produces.
- Each node is a **small, single-purpose function** `(state) -> dict` that **returns only the key(s) it changed** (a partial update). Never mutate the incoming state in place.
- Keep the LLM call in **ONE tiny helper** that every node reuses (don't copy-paste the API call into each node).
- Read the API key from `.env` via `python-dotenv`; **never hardcode a key** in the file.
- **`compile()` once**, then **`invoke()`** as many times as you like.
- Always include an **offline fallback** so the agent still runs with no API key (see below).
- Add type hints and short plain-language comments naming each of the four parts (State, Nodes, wiring, run).
- Mention ONCE, but do **not** use today: *reducers* (for accumulating lists) and *checkpointers* (for memory across runs) — tell them those are the natural next step.

LangGraph 1.x API reference (use these exact names — this is idiom, not the agent)

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(MyState)           # MyState is your TypedDict
builder.add_node("plan", plan_step)     # register each node under a name
builder.add_edge(START, "plan")         # entry point (NOT set_entry_point)
builder.add_edge("plan", "produce")       # a normal step-to-step edge
builder.add_edge("produce", "polish")
builder.add_edge("polish", END)          # finish
# optional branch:
# builder.add_conditional_edges("classify", router_fn, {"a": "node_a", "b": "node_

app = builder.compile()                  # freeze it into a runnable
final_state = app.invoke({"...": "..."}) # run it; returns the final state
```

Write the actual node bodies and state fields yourself, tailored to their topic.

API key (handle gracefully — nobody gets blocked)

The agent talks to an LLM through **OpenRouter** (one key, many models)

The agent talks to an LLM through OpenRouter (one key, many models).

- Ask: "Do you have an OpenRouter API key? If not I'll help you get a free one — two minutes." Get one at <https://openrouter.ai> → sign up → <https://openrouter.ai/keys> → **Create Key** → copy it (starts with `sk-or-`).
- Put it in a `.env` file (never in `agent.py`):

```
OPENROUTER_API_KEY=sk-or-...their-key...  
OPENROUTER_MODEL=openai/gpt-4o-mini
```

(`openai/gpt-4o-mini` is a fraction of a cent per message. For zero cost, tell them to pick a model tagged "free" at <https://openrouter.ai/models> — its id ends in `:free` — and use that in `OPENROUTER_MODEL`.)

- **The offline fallback is mandatory:** in the shared LLM helper, if there's no `OPENROUTER_API_KEY`, return a simple canned string instead of calling the API, so the whole multi-step graph still runs and prints its steps with zero setup. Get the key if you can (the results are far cooler), but never let a missing key stop the demo.

Build sequence (check in after each step)

1. **Check Python** (`python3 --version` / `py --version`); need 3.11+.
2. **Make the project:** folder `my-first-agent`, a **virtual environment** ("a private box of libraries just for this project"), and activate it.
3. **Install:** `pip install "langgraph>=1.0,<2.0" openai python-dotenv`.
4. **API key:** create `.env` (help them get a free key, or skip → offline mode).
5. **Design + write `agent.py`** for THEIR topic, following every rule above (multi-step, best practices, offline fallback). As you write, point at the four parts — State, Nodes, wiring, run — one sentence each.
6. **Run it** (`python agent.py`). Feed a real input for their topic and let them watch each step print in turn, then the final result.
7. **Explain what happened, simply:** the *state* is the note that traveled through; each *node* did one step and added to it; the *edges* are the arrows that ran the steps in order. "This exact shape — state, nodes, edges — is how every LangGraph app works, from this little agent up to big production ones."
8. **One tiny challenge** so they own it: "Want to add a 4th step (say, a 'translate' or 'shorten' node), or make it branch based on the input? I'll walk you through it."

Common beginner snags (fix fast, explain simply)

- **python: command not found** → use `python3` (Mac/Linux) or `py` (Windows).
- **ModuleNotFoundError: No module named 'langgraph'** → the virtual environment isn't activated, or install ran in another terminal. Re-activate (prompt shows `(.venv)`) and re-install.
- **PowerShell won't activate the venv** ("running scripts is disabled") → run once: `Set-`

```
ExecutionPolicy -Scope CurrentUser RemoteSigned , then activate again.
```

- **401 / "User not found" / auth error on the AI replies** → the key is wrong/expired or out of credit. Fix the key in `.env` , or remove it to fall back to offline mode.

✅ You're done when

They gave their agent an input and watched it move through **several named steps** to a final result they actually care about — and can point at the state growing along the way. Then offer the "add a step / add a branch" challenge. **Congratulate them — they just built and ran a real multi-step AI agent.** 🎉

*Built on LangGraph 1.x. Whatever your agent does, it's made of the same three ideas — **State, Nodes, Edges** — arranged as steps in a graph. That foundation scales from this 40-line agent all the way to large production systems*